### B. Separation of Mean Control and Covariance Control

In this section, we will separate Problem 2 into a mean control problem and a covariance control problem.

By defining the augmented vectors $U_k = [u_0^\top \ u_1^\top \ \cdots \ u_k^\top]^\top$, $\Xi_k = [\xi_0^\top \ \xi_1^\top \ \cdots \ \xi_k^\top]^\top$, and $\hat{X}_k = [\hat{x}_0^\top \ \hat{x}_1^\top \ \cdots \ \hat{x}_k^\top]^\top$. We can compute $\hat{X}_k$ as follows

$$\hat{X} = A\hat{x}_{0^-} + BU + H + L\Xi, \tag{12}$$

where $\hat{X} = \hat{X}_N$, $U = U_{N-1}$, $\Xi = \Xi_N$, and $A$, $B$, $H$, and $L$ are block matrices constructed using the system matrices in (10). Defining $\check{x}_{0^-} \triangleq \hat{x}_{0^-} - \bar{x}_0$, $\bar{X} \triangleq \mathbb{E}[\hat{X}]$, $\bar{U} \triangleq \mathbb{E}[U]$, $\check{X} \triangleq \hat{X} - \bar{X}$, and $\tilde{U} \triangleq U - \bar{U}$, and using (12), it follows that

$$\bar{X} = A\bar{x}_0 + B\bar{U} + H, \tag{13}$$

$$\check{X} = A\check{x}_{0^-} + B\tilde{U} + L\Xi. \tag{14}$$

The cost functional in (11) can be rewritten as

$$\begin{aligned}\hat{J}(u) =& \mathbb{E}[\check{X}^\top Q\check{X} + \tilde{U}^\top R\tilde{U}] + \bar{X}^\top Q\bar{X} + \bar{U}^\top R\bar{U} \\ & - 2\bar{X}^\top QM_r,\end{aligned} \tag{15}$$

where $Q = \text{blkdiag}(Q_0, Q_1, \ldots, Q_{N-1}, 0)$, $R = \text{blkdiag}(R_0, R_1, \ldots, R_{N-1})$, $M_r = [m_0^\top \ m_1^\top \ \cdots \ m_N^\top]^\top$.

From (13), (14), and (15), the covariance steering problem of the estimated state $\hat{x}$ (Problem 2) can thus be divided into the following mean control and covariance control problems. The mean control problem is given by

$$\begin{aligned}\min_{\bar{U}} \quad & \bar{X}^\top Q\bar{X} + \bar{U}^\top R\bar{U} - 2\bar{X}^\top QM_r \\ \text{s.t.} \quad & \bar{X} = A\bar{x}_0 + B\bar{U} + H, \\ & E_0\bar{X} = \bar{x}_0, \quad E_N\bar{X} = \bar{x}_N,\end{aligned} \tag{16}$$

where $E_k$ is a matrix defined such that $E_k\bar{X} = \bar{x}_k$. The covariance control problem is given by

$$\begin{aligned}\min_{\tilde{U}} \quad & \mathbb{E}[\check{X}^\top Q\check{X} + \tilde{U}^\top R\tilde{U}] \\ \text{s.t.} \quad & \check{X} = A\check{x}_{0^-} + B\tilde{U} + L\Xi, \\ & \mathbb{E}[\check{x}_{0^-}\check{x}_{0^-}^\top] = \hat{P}_{0^-}, \quad E_N\mathbb{E}[\check{X}\check{X}^\top]E_N^\top = \hat{P}_N,\end{aligned} \tag{17}$$

where $\hat{P}_{0^-} = P_0 - \tilde{P}_{0^-}$, $\hat{P}_N = P_N - \tilde{P}_N \preceq P_f - \tilde{P}_N$.

### C. Solutions of the Mean Control and Covariance Control Problems

Assuming that the system (3) is controllable, the solution for the mean control problem (16) to obtain the mean trajectory $(\bar{x}_k)_{k=0}^N$ and $(\bar{u}_k)_{k=0}^{N-1}$ is given by [25]

$$\begin{aligned}\bar{U}^* =& W(-V + \bar{B}_N^\top(\bar{B}_N W\bar{B}_N^\top)^{-1} \\ & (\bar{x}_N - \bar{A}_N\bar{x}_0 - H_N + \bar{B}_N WV)),\end{aligned} \tag{18}$$

where $W = (B^\top QB + R)^{-1}$, and $V = B^\top Q(A\bar{x}_0 + H - M_r)$.

The solution to the covariance control problem (17) is given by the controller of the form

$$\tilde{u}_k = \sum_{i=0}^{k} K_{k,i}\check{x}_i, \tag{19}$$

equivalently, by $\tilde{U} = K\check{X}$, where the control gain matrix $K$ is lower block diagonal of the form

$$K = \begin{bmatrix} K_{0,0} & 0 & \cdots & 0 & 0 \\ K_{1,0} & K_{1,1} & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & 0 \\ K_{N-1,0} & K_{N-1,1} & \cdots & K_{N-1,N-1} & 0 \end{bmatrix}, \tag{20}$$

and is computed from $K = F(I+BF)^{-1}$ where $F$ is obtained by solving the following convex optimization problem [27]

$$\min_F \ \text{trace}[((I+BF)^\top Q(I+BF) + F^\top RF)P_Z], \tag{21}$$

subject to

$$\|P_Z^{1/2}(I+BF)^\top E_N^\top(P_f - \tilde{P}_N)^{-1/2}\| - 1 \leq 0, \tag{22}$$

where $P_Z = A\hat{P}_{0^-}A^\top + LP_\Xi L^\top$ is the covariance of $Z \triangleq A\check{x}_{0^-} + L\Xi$ and $P_\Xi = \text{blkdiag}(P_{\xi_0}, \cdots, P_{\xi_N})$ is the covariance of the innovation process.

## IV. Computing the Nominal Trajectory

In this section, we develop an efficient algorithm to find nominal trajectories for nonlinear systems using the mean controller (18). A deterministic nonlinear dynamic model, i.e., with the noise $w_k$ set to zero, is considered when designing the nominal trajectory. To this end, consider the deterministic nonlinear system

$$x_{k+1} = f(x_k, u_k, 0). \tag{23}$$

Let $(x_k^r)_{k=0}^{N-1}$ and $(u_k^r)_{k=0}^{N-1}$ be a nominal trajectory for this system. The linearized model (3) along this nominal trajectory will be used by the mean controller (18) to compute a control sequence $(u_k^c)_{k=0}^{N-1}$ and the corresponding state sequence $(x_k^c)_{k=0}^{N-1}$.

*Definition 1:* A nominal trajectory is called a *compatible nominal trajectory* for system (23) if $x_k^r = x_k^c$ and $u_k^r = u_k^c$ for $k = 0, \ldots, N-1$.

Linearizing along a compatible nominal trajectory ensures that the nonlinear system is linearized at the "correct" points. When applying the designed control to the linearized system, the resulting trajectory is the same as the nominal trajectory, which means that the system reaches the exact points where the linearization is performed. On the other hand, there will be extra errors caused by the linearization if the system is linearized along a non-compatible nominal trajectory.

The iterative algorithm to find a compatible nominal trajectory is given in Algorithm 1.

When the algorithm converges, the nominal trajectory is the same as the mean optimal trajectory, which, by definition, is a compatible nominal trajectory. Note that (18) gives the analytical solution of the mean control, which can be quickly computed. Each iteration of the algorithm has a low computational load and the overall algorithm may be solved efficiently. As seen later in Section VI, the algorithm typically convergences within a few iterations.

**Algorithm 1:** Compatible Nominal Trajectory (CNT)

Initialize $(x_k^r)_{k=0}^{N-1}$ and $(u_k^r)_{k=0}^{N-1}$ ;
**while** *NotConverged* **do**
  Linearize (23) along $(x_k^r)_{k=0}^{N-1}$ and $(u_k^r)_{k=0}^{N-1}$ ;
  Compute the mean optimal control using (18) to obtain the control sequence $(u_k^c)_{k=0}^{N-1}$ ;
  Compute the the controlled state trajectory $(x_k^c)_{k=0}^{N-1}$ using (13) ;
  Set $(x_k^r = x_k^c)_{k=0}^{N-1}$ and $(u_k^r = u_k^c)_{k=0}^{N-1}$ ;
**return** $(x_k^r)_{k=0}^{N-1}$ and $(u_k^r)_{k=0}^{N-1}$;

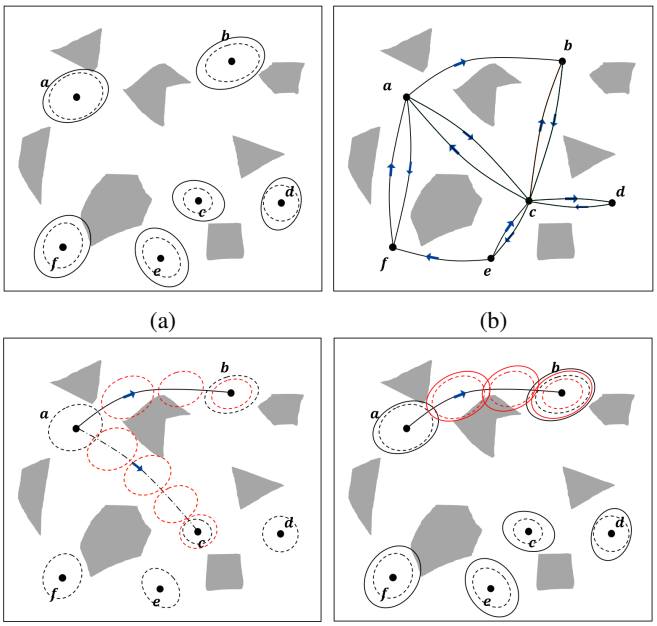


Fig. 1: Construction of the CS-BRM. Gray shapes denote obstacles. (a) Node sampling. Each node is represented by its mean, a state covariance, and a state estimation error covariance, all of which are sampled in proper sample spaces; (b) Mean trajectories are computed for all neighboring node pairs using the mean controller. Only mean trajectories that are collision-free are preserved. Those mean trajectories indicate possible connections/edges in the CS-BRM; (c) Kalman filter updates are simulated for each possible edge; (d) Covariance control is applied for each edge to execute the transition between the nodes.

## V. The CS-BRM Algorithm

The nodes in the CS-BRM are sampled in the belief space. In a partially observable environment, the belief $b_k$ at time-step $k$ is given by the conditional probability distribution of the state $x_k$, conditioned on the history of observations $(y_i)_{i=0}^{k}$ and the history of control inputs $(u_i)_{i=0}^{k-1}$, that is, $b_k = \mathbb{P}(x_k|(y_i)_{i=0}^{k}, (u_i)_{i=0}^{k-1})$. In a Gaussian belief space, $b_k$ can be equivalently represented by the estimated state, $\hat{x}_k$, and the estimation error covariance, $\tilde{P}_k$, that is, $b_k = (\hat{x}_k, \tilde{P}_k)$ [18, 14]. The state estimate $\hat{x}_k$ is Guassian, and is given by $\hat{x}_k \sim \mathcal{N}(\bar{x}_k, \hat{P}_k)$. Hence, the Gaussian belief can be also written as $b_k = (\bar{x}_k, \hat{P}_k, \tilde{P}_k)$. The main idea of CS-BRM is to use covariance steering theory to design the edge controller to achieve node reachability.

**Algorithm 2:** Constructing CS-BRM

1 $V = \{nod_1, \ldots, nod_n\} \leftarrow$ `SampleNodes`$(n)$ ;
2 $V_r \leftarrow V$, $E \leftarrow \emptyset$ ;
3 **for** $i = 1 : n$ **do**
4  $V_{near} \leftarrow$ `Neighbor`$(V_r,\ nod_i)$;
5  **foreach** $nod_j \in V_{near}$ **do**
6  $(\bar{U}_{ij},\ \tau_{ij},\ MCost_{ij}) \leftarrow$ `MTraj`$(nod_i,\ nod_j)$ ;
7  **if** `ObstacleFree`$(\tau_{ij})$ **then**
8  $\tilde{P}_{N^-} \leftarrow$ `KF`$(nod_i,\ nod_j)$ ;
9  **if** $\tilde{P}_{N^-} \preceq \tilde{P}_{nod_j}$ **then**
10  $(\tilde{U}_{ij},\ CovCost_{ij}) \leftarrow$ `CovControl`$(nod_i,\ nod_j)$ ;
11  $CollisionCost_{ij} \leftarrow$ `MonteCarlo`$(\bar{U}_{ij},\ \tilde{U}_{ij})$ ;
12  $E_{ij} \leftarrow (\bar{U}_{ij},\ \tilde{U}_{ij},\ EdgeCost_{ij})$ ;
13  $E \leftarrow E \cup E_{ij}$
14  $(\bar{U}_{ji},\ \tau_{ji},\ MCost_{ji}) \leftarrow$ `MTraj`$(nod_j,\ nod_i)$ ;
15  **if** `ObstacleFree`$(\tau_{ji})$ **then**
16  Repeat line 8-13 with $i$ and $j$ swapped
17  $V_r \leftarrow V_r \setminus nod_i$ ;
18 CS-BRM $\leftarrow (V,\ E)$ ;
19 **return** CS-BRM;

An illustration of the steps for building the CS-BRM is shown in Fig. 1. The algorithm for constructing the CS-BRM is given in Algorithm 2. The following procedures are used in the algorithm.

**Sample Nodes:** The function `SampleNodes`$(n)$ samples $n$ CS-BRM nodes. A node in the CS-BRM is represented by the tuple $(\bar{x}, \hat{P}_{\text{-}}, \tilde{P}_{\text{-}})$, where $\bar{x}$ is the state mean, $\hat{P}_{\text{-}}$ is the prior estimated state covariance, and $\tilde{P}_{\text{-}}$ is the prior state estimation error covariance. Since $P = \hat{P}_{\text{-}} + \tilde{P}_{\text{-}}$, the node can also be equivalently represented by $(\bar{x}, P, \tilde{P}_{\text{-}})$. Note that we can compute the a posteriori convariances $\hat{P}$ and $\tilde{P}$ using $\hat{P}_{\text{-}}$ and $\tilde{P}_{\text{-}}$. For constructing node $j$, the algorithm starts by sampling the free state space, which provides the mean $\bar{x}_j$ of the distribution. Then, an estimated state covariance $\hat{P}_{j^-}$ is sampled from the space of symmetric and positive definite matrix space. Similarly, a state estimation error covariance $\tilde{P}_{j^-}$ is also sampled at node $j$. $\tilde{P}_{j^-}$ is the initial condition of the Kalman filter update for the edges that are coming out of node $j$. In Fig. 1(a), for each node, $\bar{x}$ is shown as a black dot, $P$ is shown as a solid ellipse, and $\tilde{P}_{\text{-}}$ is shown as a dashed ellipse.

**Neighbor:** The function `Neighbor`$(V_r, nod_i)$ finds all the nodes in $V_r$ that are within a given distance $d_1$ to node $nod_i$, where $V_r$ is a node set containing all nodes in the CS-BRM.

**Mean Trajectory:** Given two nodes $nod_i$ and $nod_j$, $\texttt{MTraj}(nod_i, nod_j)$ uses the mean controller (18) and Algorithm 1 to find the compatible nominal trajectory, which is also the mean trajectory from $nod_i$ to $nod_j$. The function returns the mean control $\bar{U}_{ij}$, mean trajectory $\tau_{ij}$, and the cost of the mean control $MCost_{ij}$.

**Obstacle Checking:** The function $\texttt{ObstacleFree}(\tau_{ij})$ returns true if the mean trajectory $\tau_{ij}$ is collision free.

**Kalman Filter:** Given two nodes $nod_i$ and $nod_j$, $\texttt{KF}(nod_i,\ nod_j)$ returns the prior estimation error covariance at the last time step of the trajectory from $nod_i$ to $nod_j$.

**Covariance Control:** $\texttt{CovControl}(nod_i, nod_j)$ solves the covariance control problem from $nod_i$ to $nod_j$. It returns the control $\tilde{U}_{ij}$ and the cost $CovCost_{ij}$.

**Monte Carlo:** We use Monte Carlo simulations to calculate the probability of collision of the edges. For edge $E_{ij}$, the initial state $x_0$ and initial state estimate $\hat{x}_{0^-}$ are sampled from their corresponding distributions. The state trajectory is simulated using the mean control $\bar{U}_{ij}$ and covariance control $\tilde{U}_{ij}$. Then, collision checking is performed on the simulated state trajectory. By repeating this process, we approximate the probability of collision of this edge. In our implementation the collision cost $CollisionCost_{ij}$ is taken to be proportional to the probability of collision along that edge.

Algorithm 2 starts by sampling $n$ nodes in the belief space using $\texttt{SampleNodes}$ (Line 1). Lines 3-17 are the steps to add CS-BRM edges. Given two neighboring nodes $nod_i$ and $nod_j$, Lines 6-13 try to construct the edge $E_{ij}$ and Lines 14-16 try to construct the edge $E_{ji}$.

Every edge in the CS-BRM is constructed by solving a covariance steering problem. Each node tries to connect to its neighboring nodes if possible. First, for each edge, we use Algorithm 1 to find the mean trajectory of that edge (Line 6). Next, we check if the mean trajectory is collision-free. Only if the mean trajectory is collision-free do we proceed to the next step of edge construction. Then, the Kalman filter updates are simulated, which give the state estimation error covariance at every time step along that edge. The state estimation error covariances of the edges $\overline{ab}$ and $\overline{ac}$ are shown as red dash ellipses in Figure 1(c). Take the edge $\overline{ab}$ as an example. The initial condition of the Kalman filter is $\tilde{P}_{0^-} = \tilde{P}_{a^-}$. The prior estimation error covariance at the final time-step, $\tilde{P}_{N^-}$, is compared with the state estimation error covariance of node $b$, $\tilde{P}_{b^-}$. If $\tilde{P}_{N^-} \preceq \tilde{P}_{b^-}$ (Line 9), the algorithm proceeds to solve the covariance control problem and this edge is added to the graph. For the covariance control problem of edge $\overline{ab}$, the initial constraint and the terminal constraint are given by $\hat{P}_{0^-} = \hat{P}_{a^-}$ and $\hat{P}_N \preceq \hat{P}_b$. The covariance $\hat{P}_b$ is computed using $\hat{P}_{b^-}$ and $\tilde{P}_{b^-}$. The final edge controller is the combination of the mean controller, covariance controller, and the Kalman filter.

In addition to the edge controller, the edge cost is computed for each edge. The edge cost $EdgeCost_{ij}$ is a weighted sum of $MCost_{ij}$, $CovCost_{ij}$, and $CollisionCost_{ij}$. With covariance steering serving as the lower-level controller, the higher-level motion planning problem using the roadmap is a graph search problem similarly to a PRM. Thus, the covariance steering approach transforms the belief space roadmap into traditional PRM with specific edge costs.

In CS-BRM, each edge is an independent covariance steering problem and the planned path using CS-BRM consists of a concatenation of edges. Since the terminal estimated state covariance satisfies an inequality constraint (17) and the terminal state estimation error covariance satisfies an inequality relation by construction (Line 9), it is important to verify that the covariance constraints at all nodes are still satisfied by concatenating the edges.

To this end, consider the covariance steering problem from node $a$ to node $b$. For the edge $\overline{ab}$, we have the initial constraints $\hat{P}_{0^-} = \hat{P}_{a^-}$, $\tilde{P}_{0^-} = \tilde{P}_{a^-}$, and the terminal covariance constraint $\hat{P}_N \preceq \hat{P}_b$. Note that the constraint $\tilde{P}_{N^-} \preceq \tilde{P}_{b^-}$ is satisfied for $\overline{ab}$ (Line 9, Algorithm 2). Suppose that the solution of the covariance control problem of edge $\overline{ab}$ is the feedback gain $K$. By concatenating the edges, the system may not start exactly at node $a$. Instead, it will start at some node $a^{'} = (\bar{x}_a, \hat{P}^{'}_{a^-}, \tilde{P}^{'}_{a^-})$ that satisfies $\hat{P}^{'}_{a^-} \preceq \hat{P}_{a^-}$ and $\tilde{P}^{'}_{a^-} \preceq \tilde{P}_{a^-}$. Next, we show that by applying the pre-computed feedback gain $K$, the terminal covariance still satisfies $\hat{P}^{'}_N \preceq \hat{P}_b$ for the new covariances $\hat{P}^{'}_{a^-}$ and $\tilde{P}^{'}_{a^-}$. Before verifying this result, we provide a property of the covariance of the Kalman filter. Given an initial estimation error covariance $\tilde{P}_{0^-}$, we obtain a sequence of error covariances $(\tilde{P}_{k^-})_{k=1}^N$ using the Kalman filter updates (7). Similarly, given the new initial error covariance $\tilde{P}^{'}_{0^-}$ which satisfies $\tilde{P}^{'}_{0^-} \preceq \tilde{P}_{0^-}$, we obtain a new sequence of error covariances $(\tilde{P}^{'}_{k^-})_{k=0}^N$. We have the following lemma.

*Lemma 1:* If $\tilde{P}^{'}_{0^-} \preceq \tilde{P}_{0^-}$, then $\tilde{P}^{'}_{k^-} \preceq \tilde{P}_{k^-}$, for all $k = 0, \cdots, N$.

The proof of this lemma is omitted. A proof of a similar result can be found in [18]. From Lemma 1, it is straightforward to show that, if $\tilde{P}^{'}_{0^-} \preceq \tilde{P}_{0^-}$, we also have $\tilde{P}^{'}_k \preceq \tilde{P}_k$ for all $k = 0, \ldots, N$.

*Proposition 2:* Consider a path on the CS-BRM roadmap, where the initial node of the path is denoted by $(\bar{x}_i, \hat{P}_i, \tilde{P}_i)$ and the final node of the path is denoted by $(\bar{x}_j, \hat{P}_j, \tilde{P}_j)$. Starting from the initial node and following this path by applying the sequence of edge controllers, the robot will arrive at a belief node $(\bar{x}, \hat{P}, \tilde{P})$ such that $\bar{x} = \bar{x}_j$, $\hat{P} \preceq \hat{P}_j$, and $\tilde{P} \preceq \tilde{P}_j$.

The proof of this proposition is straightforward and is omitted. Proposition 2 guarantees that, when planning on the CS-BRM roadmap, the covariance at each arrived node is always smaller than the assigned fixed covariance at the corresponding node of the CS-BRM roadmap.

## VI. Numerical Examples

In this section, we first illustrate our theoretical results for the motion planning problem of a 2-D double integrator. We compare our method with the SLQG-FIRM algorithm [14], and we show that our method overcomes some of the limitations of SLQG-FIRM. Subsequently, the problem of a fixed-wing aerial vehicle in a 3-D environment is studied.

### A. 2-D Double Integrator

A 2-D double integrator is a linear system with a 4-dimensional state space given by its position and velocity, and